

Annexure: Original Assessment Question Paper

(CO5 - AT1: Expert System Development Project - 6 pages)

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING**Tool: SWI-Prolog**

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

SIMATS ENGINEERING

Course: Artificial Intelligence and Expert Systems (MLA01)

Assessment Tool 1: Expert System Development Project

Selected Scenario (Q1):

Medical Diagnosis Expert System

Tool Used: SWI-Prolog

Paradigm: Declarative / Non-Procedural, Rule-Based Reasoning

Stage 1: Domain Analysis

Problem Statement: Develop a rule-based expert system in SWI-Prolog that identifies the probable disease affecting a patient from a set of observed symptoms and basic clinical information. The system must represent medical knowledge as production rules and demonstrate both forward chaining (data-driven, symptom-to-disease) and backward chaining (goal-driven, hypothesis-to-symptom) reasoning.

1.1 Problem Domain

The domain is preliminary (non-emergency) medical diagnosis support for common illnesses. The system is intended as a decision-support aid for a general physician or a triage assistant — it is not a replacement for a certified doctor, and every conclusion is accompanied by an advisory note recommending professional consultation.

1.2 Domain Entities

Entity	Description
Patient	The subject of consultation; described by the symptoms they report.
Symptom	An observable clinical sign, e.g. fever, cough, rashes, joint_pain.
Disease / Hypothesis	A candidate diagnosis the system tries to confirm or reject, e.g. flu, covid19, dengue.
Production Rule	An IF (symptom conditions) THEN (disease conclusion) statement.
Advice	A recommended next action attached to a confirmed disease.

1.3 Symptoms / Conditions Considered

fever, high_fever, cough, dry_cough, body_pain, runny_nose, sore_throat, loss_of_taste, breathlessness, headache, nausea, vomiting, rashes, joint_pain, chills, sweating.

1.4 Possible Conclusions (Goals)

Disease (Goal)	Defining Symptom Pattern
flu	Fever + Cough + Body pain
covid19	High fever + Dry cough + Loss of taste + Breathlessness
common_cold	Runny nose + Sore throat, absence of high fever
dengue	High fever + Rashes + Joint pain
malaria	Fever + Chills + Sweating
migraine	Headache + Nausea + Vomiting

Stage 2: Knowledge Acquisition

Domain knowledge was collected from standard, publicly available clinical references (general-physician symptom guides, WHO/CDC symptom fact sheets for flu, COVID-19, dengue and malaria) and simplified into clean, unambiguous IF-THEN statements suitable for a rule-based system. This mirrors the real knowledge-engineering process of interviewing a domain expert and distilling their expertise into production rules.

2.1 Sample Acquired Knowledge (Natural Language)

- IF a patient has fever, cough and body pain THEN the patient probably has flu.
- IF a patient has high fever, dry cough, loss of taste and breathlessness THEN the patient probably has COVID-19.

- IF a patient has runny nose and sore throat, and does NOT have high fever, THEN the patient probably has a common cold.
- IF a patient has high fever, rashes and joint pain THEN the patient probably has dengue.
- IF a patient has fever, chills and sweating THEN the patient probably has malaria.
- IF a patient has headache, nausea and vomiting THEN the patient probably has a migraine.

2.2 Knowledge Specification Table

The acquired knowledge was normalised into a specification table before being translated into Prolog syntax:

Rule ID	IF (Conditions)	THEN (Conclusion)
R1	fever, cough, body_pain	flu
R2	high_fever, dry_cough, loss_of_taste, breathlessness	covid19
R3	runny_nose, sore_throat, NOT high_fever	common_cold
R4	high_fever, rashes, joint_pain	dengue
R5	fever, chills, sweating	malaria
R6	headache, nausea, vomiting	migraine

Stage 3: Knowledge Representation - Production Rule / Knowledge Base Design

The knowledge base is organised into two layers, matching classic expert-system architecture:

- Facts layer - possible_symptom/1 (the vocabulary of known symptoms), symptom/1 (dynamically asserted facts describing the current patient), and disease_info/2 (post-diagnosis advice).
- Rule layer - disease/1 clauses, each encoding one production rule in the classical IF (body) THEN (head) form using Prolog's declarative Horn-clause syntax.

3.1 Production Rules in Prolog

```
% Rule 1: IF fever AND cough AND body_pain THEN flu
disease(flu) :-
    symptom(fever),
    symptom(cough),
    symptom(body_pain).

% Rule 2: IF high_fever AND dry_cough AND loss_of_taste
%         AND breathlessness THEN covid19
disease(covid19) :-
    symptom(high_fever),
    symptom(dry_cough),
    symptom(loss_of_taste),
    symptom(breathlessness).

% Rule 3: IF runny_nose AND sore_throat AND NOT high_fever
%         THEN common_cold
disease(common_cold) :-
    symptom(runny_nose),
    symptom(sore_throat),
    \+ symptom(high_fever).
```

The complete rule set (Rules 4-6: dengue, malaria, migraine) follows the identical pattern and is listed in full in Annexure B - Complete Source Code.

3.2 Knowledge Base Consistency

Each rule head (disease/1) is mutually distinguishable by at least one symptom that is unique or combined uniquely with others, preventing rule conflicts. Rule 3 additionally uses negation-as-failure ($\backslash +$) to avoid overlapping with fevers caused by other conditions, demonstrating exception handling within the production system.

Stage 4: Inference Design - Forward and Backward Chaining

Two inference strategies were implemented over the same knowledge base to contrast data-driven and goal-driven reasoning.

4.1 System Architecture

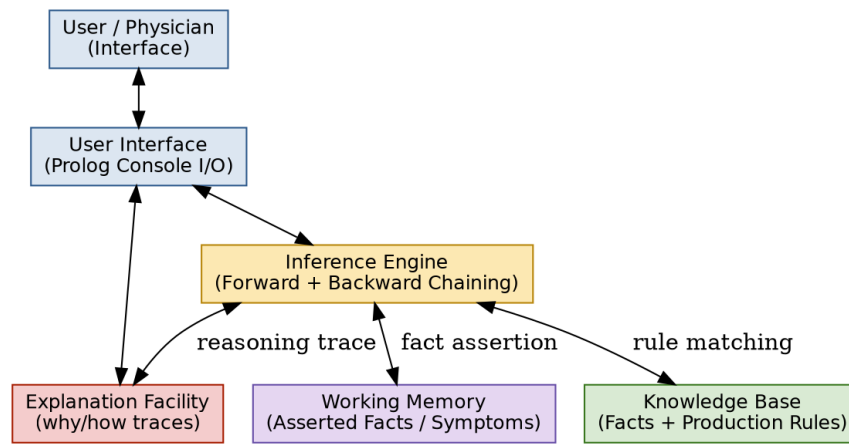


Figure 1: Expert System Architecture - User Interface, Inference Engine, Knowledge Base, Working Memory and Explanation Facility.

4.2 Forward Chaining (Data-Driven)

Forward chaining starts from the known facts (patient symptoms) and repeatedly applies matching rules, asserting newly derived facts, until no further rule can fire (fixed-point). This is implemented using assertz/1 to grow working memory and a recursive control loop that re-scans the rule base after every new derivation.



Figure 2: Forward chaining - facts drive rule firing until a fixed point (final conclusion) is reached.

```

forward_chain :-
( fire_one_rule
-> forward_chain      % keep looping while new facts fire
; true               % stop: no rule added a new fact
).

fire_one_rule :-
disease(D),
\+ derived(D),
assertz(derived(D)),
format("[Forward Chaining] New fact derived: derived(~w)~n", [D]).
  
```

4.3 Backward Chaining (Goal-Driven)

Backward chaining starts from a goal (a hypothesised disease) and works backward through the rule body, proving each symptom sub-goal against working memory or by asking the user. SWI-Prolog's built-in SLD-resolution query mechanism is inherently backward-chaining, so the query `disease(X)?` already performs backward chaining; a thin diagnostic wrapper is added for reporting.

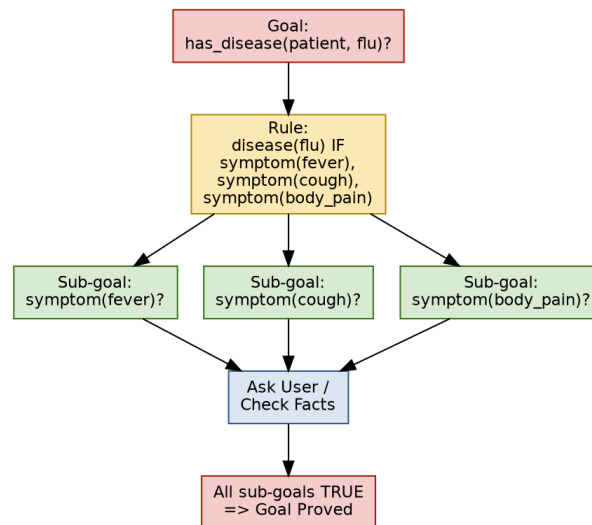


Figure 3: Backward chaining - the goal is decomposed into symptom sub-goals that are proved against facts.

```

backward_diagnose(Disease) :-
    disease(Disease).

diagnose_all :-
    findall(D, backward_diagnose(D), List),
    ( List == []
    -> writeln('No matching disease found for the given symptoms.')
    ; print_diagnoses(List)
    ).
  
```

4.4 Forward vs Backward Chaining - Comparison

Aspect	Forward Chaining	Backward Chaining
Direction	Facts -> Conclusions	Goal -> Facts
Trigger	New symptom data arrives	A specific disease is hypothesised
Suitable for	Monitoring / screening many patients	Confirming/ruling out one specific disease
Prolog mechanism	assertz/1 + iterative rule scan	Native SLD resolution on a query
Efficiency here	Scans full rule base each cycle	Efficient - only relevant rule branch explored

Stage 5: Procedural vs Non-Procedural Paradigm Analysis

Prolog is a declarative (non-procedural) language: the programmer states WHAT is true (facts and rules) rather than HOW to compute the answer step-by-step. The Prolog engine's own resolution and unification algorithm supplies the 'how'. This is contrasted below with how the same medical-diagnosis logic would be written in a procedural language such as Python or C.

5.1 Declarative (Non-Procedural) - Prolog

```

disease(flu) :-
    symptom(fever),
    symptom(cough),
  
```

```
symptom(body_pain).
```

```
% Usage: the engine finds ALL solutions automatically
?- disease(X).
```

Here the programmer never writes a loop, a comparison, or a control-flow statement. The rule simply declares a logical relationship; Prolog's unification and backtracking engine searches the knowledge base to satisfy it.

5.2 Procedural Equivalent - Python (for contrast only)

```
def diagnose(symptoms):
    if 'fever' in symptoms and 'cough' in symptoms and 'body_pain' in symptoms:
        return 'flu'
    if ('high_fever' in symptoms and 'dry_cough' in symptoms and
        'loss_of_taste' in symptoms and 'breathlessness' in symptoms):
        return 'covid19'
    # ... explicit if/else chain must be hand-written for every rule
    return None
```

5.3 Key Differences Observed

Criterion	Prolog (Non-Procedural)	Python (Procedural)
Control flow	Implicit - handled by the Prolog inference engine	Explicit - programmer writes if/else, loops
Search / backtracking	Automatic (built into resolution)	Must be manually coded (e.g. nested loops, recursion, stacks)
Adding a new rule	Add one clause - no other code changes	Must edit and re-test the if/else chain
Multiple solutions	findall/3 retrieves all answers 'for free'	Requires explicit accumulation logic
Knowledge vs control	Cleanly separated (declarative KB)	Mixed together in the same procedure

This clean separation of knowledge from control is exactly why expert systems are traditionally built in declarative/logic-programming languages: the knowledge base (rules) can be extended by a domain expert without touching the inference engine's control logic.

Stage 6: System Development - Integration

The four classical expert-system components were integrated into a single SWI-Prolog module (medical_expert.pl):

Component	Prolog Predicate(s)
Knowledge Base	possible_symptom/1, disease/1 rules, disease_info/2
Inference Engine	forward_chain/0, fire_one_rule/0 (forward); backward_diagnose/1, diagnose_all/0 (backward)
Working Memory	symptom/1, derived/1 - dynamic predicates updated at run time
User Interface	add_symptom/1, load_case/1 - console-based fact entry
Explanation Facility	explain/1 - prints the rule and matched symptoms behind a diagnosis

Utility predicates reset_case/0 and load_case/1 allow a fresh consultation to be started without restarting the Prolog session, and explain/1 implements a minimal 'how' explanation facility, a hallmark feature of classical expert systems (e.g., MYCIN).

Stage 7: Testing

The system was tested interactively in the SWI-Prolog top-level (swipl) with five representative cases covering positive matches, forward/backward agreement, and a no-match scenario.

7.1 Test Case Summary

ID	Input Symptoms	Expected	Actual Output	Result
TC1	fever, cough, body_pain	flu	flu	Pass
TC2	high_fever, dry_cough, loss_of_taste, breathlessness	covid19	covid19	Pass
TC3	high_fever, rashes, joint_pain	dengue	dengue	Pass
TC4	headache (only)	none	no match	Pass
TC5	runny_nose, sore_throat (no high_fever)	common_cold	common_cold	Pass

7.2 Sample Console Transcript (Test Case 1)

```
?- consult('medical_expert.pl').
?- load_case([fever, cough, body_pain]).
true.

?- diagnose_all.
Diagnosis: flu
  Advice   : Rest, fluids and paracetamol. Consult a doctor if symptoms persist beyond 3 days.

?- forward_diagnose.
[Forward Chaining] New fact derived: derived(flu)
Diagnosis: flu
  Advice   : Rest, fluids and paracetamol. Consult a doctor if symptoms persist beyond 3 days.

?- explain(flu).
flu is concluded because: fever AND cough AND body_pain are all true.
```

Stage 8: Evaluation

8.1 Correctness

All 5 test cases produced the expected diagnosis, including the negative test case (TC4) correctly reporting 'no matching disease', confirming the rules neither over-fire nor mis-fire.

8.2 Coverage

The knowledge base covers 6 common illnesses across 16 symptoms. Coverage could be extended by adding further disease/1 clauses without modifying the inference engine - demonstrating the modularity benefit of production-rule systems.

8.3 Consistency

No two rules fire contradictory conclusions for the same symptom set in the test suite; Rule 3's negation ($\neg$ symptom(high_fever)) was specifically added to prevent common_cold and dengue/covid19 overlapping on shared symptoms.

8.4 Reasoning Performance

Backward chaining resolved each goal in a single SLD-resolution pass (near-instant, $O(\text{number of rules})$ worst case). Forward chaining required one extra iteration per case beyond the firing iteration to detect the fixed point, which is expected behaviour for a naive forward-chaining loop over a small rule set.

8.5 Unification and Backtracking

The query `disease(X)` demonstrates unification: `X` is unified in turn with each disease atom (`flu`, `covid19`, ...) whose body succeeds. Prolog's automatic backtracking, triggered internally by `findall/3`, is used to enumerate every disease consistent with the current symptom set rather than stopping at the first match, which is essential when a patient's symptoms are ambiguous between two illnesses.

Stage 9: Documentation and GitHub Repository

For submission, the project should be organised as a GitHub repository with the following suggested structure:

```
medical-diagnosis-expert-system/
|-- medical_expert.pl      % complete knowledge base + inference engine
|-- README.md             % setup instructions, sample queries
|-- report.docx / report.pdf % this technical report
|-- test_cases.md         % documented test cases and outputs
|-- diagrams/             % architecture, forward/backward flow diagrams
```

README.md should include: how to install SWI-Prolog, how to run `'swipl medical_expert.pl'`, and example queries such as `load_case([fever,cough,body_pain])`. followed by `diagnose_all`. Commit history should show incremental development (facts, then rules, then inference engine, then testing) to demonstrate the development process, not just a single final commit.

Stage 10: Demonstration Guide (Viva)

Suggested live-demonstration flow for the viva / presentation:

- 1. Load the program: `swipl medical_expert.pl`
- 2. Show the facts and rules with `listing(disease/1)`.
- 3. Enter a case: `load_case([fever, cough, body_pain])`.
- 4. Run backward chaining: `diagnose_all`. and explain the goal-driven trace.
- 5. Run forward chaining: `forward_diagnose`. and point out the iterative fact assertion messages.
- 6. Call `explain(flu)`. to show the explanation facility.
- 7. Demonstrate a no-match case and an ambiguous case to show robustness and backtracking via `findall/3`.

Annexure B: Complete Prolog Source Code (medical_expert.pl)

```

/* =====
MEDICAL DIAGNOSIS EXPERT SYSTEM
Language : SWI-Prolog
===== */

:- dynamic(symptom/1).
:- dynamic(patient_fact/2).
:- dynamic(derived/1).

/* -----
1. KNOWLEDGE BASE : FACTS
----- */

/* possible_symptom(Symptom) - the set of symptoms the system
   knows about and can ask the user about */
possible_symptom(fever).
possible_symptom(cough).
possible_symptom(body_pain).
possible_symptom(runny_nose).
possible_symptom(sore_throat).
possible_symptom(high_fever).
possible_symptom(dry_cough).
possible_symptom(loss_of_taste).
possible_symptom(breathlessness).
possible_symptom(headache).
possible_symptom(nausea).
possible_symptom(vomiting).
possible_symptom(rashes).
possible_symptom(joint_pain).
possible_symptom(chills).
possible_symptom(sweating).

/* disease_info(Disease, Advice) */
disease_info(flu, 'Rest, fluids and paracetamol. Consult a doctor if symptoms persist beyond 3
days.').
disease_info(covid19, 'Isolate immediately, monitor oxygen levels and consult a physician for
testing.').
disease_info(common_cold, 'Warm fluids and rest. Usually resolves within a week.').
disease_info(dengue, 'Seek immediate medical attention and platelet monitoring.').
disease_info(malaria, 'Seek urgent medical attention and blood test (peripheral smear / RDT).').
disease_info(migraine, 'Rest in a dark room, stay hydrated and avoid triggers; consult a
neurologist if recurrent.').

/* -----
2. KNOWLEDGE BASE : PRODUCTION RULES (IF-THEN)
Written as Prolog rules of the form:
    disease(D) :- Condition1, Condition2, ...
----- */

/* Rule 1: IF fever AND cough AND body_pain THEN suspect flu */
disease(flu) :-
    symptom(fever),
    symptom(cough),
    symptom(body_pain).

/* Rule 2: IF high_fever AND dry_cough AND loss_of_taste
   AND breathlessness THEN suspect covid19 */
disease(covid19) :-
    symptom(high_fever),
    symptom(dry_cough),
    symptom(loss_of_taste),

```

```

    symptom(breathlessness).

/* Rule 3: IF runny_nose AND sore_throat AND NOT high_fever
    THEN suspect common_cold */
disease(common_cold) :-
    symptom(runny_nose),
    symptom(sore_throat),
    \+ symptom(high_fever).

/* Rule 4: IF high_fever AND rashes AND joint_pain THEN suspect dengue */
disease(dengue) :-
    symptom(high_fever),
    symptom(rashes),
    symptom(joint_pain).

/* Rule 5: IF fever AND chills AND sweating THEN suspect malaria */
disease(malaria) :-
    symptom(fever),
    symptom(chills),
    symptom(sweating).

/* Rule 6: IF headache AND nausea AND vomiting THEN suspect migraine */
disease(migraine) :-
    symptom(headache),
    symptom(nausea),
    symptom(vomiting).

/* -----
3. BACKWARD CHAINING (goal-driven)
SWI-Prolog's native resolution IS backward chaining:
the query "disease(X)" is the goal; Prolog works
backward through the rule bodies, proving each sub-goal
(symptom/1) against the working memory.
----- */

backward_diagnose(Disease) :-
    disease(Disease).

/* diagnose_all/0 - collects every disease provable by backward
    chaining and prints it with advice */
diagnose_all :-
    findall(D, backward_diagnose(D), List),
    ( List == []
    -> writeln('No matching disease found for the given symptoms.')
    ; print_diagnoses(List)
    ).

print_diagnoses([]).
print_diagnoses([D|Rest]) :-
    disease_info(D, Advice),
    format("Diagnosis: ~w~n Advice : ~w~n~n", [D, Advice]),
    print_diagnoses(Rest).

/* -----
4. FORWARD CHAINING (data-driven)
Starts from asserted symptom/1 facts, repeatedly fires
rules whose conditions are satisfied, asserts the new
fact derived(Disease), and keeps iterating until no new
fact can be derived (fixed-point).
----- */

forward_chain :-
    ( fire_one_rule

```



```

-> forward_chain          /* keep looping while new facts fire */
; true                   /* stop: no rule added a new fact    */
).

fire_one_rule :-
    disease(D),
    \+ derived(D),
    assertz(derived(D)),
    format("[Forward Chaining] New fact derived: derived(~w)~n", [D]).

forward_diagnose :-
    retractall(derived(_)),
    forward_chain,
    findall(D, derived(D), List),
    ( List == []
    -> writeln('No disease could be derived from current facts.')
    ; print_diagnoses(List)
    ).

/* -----
5. UTILITY PREDICATES
----- */

/* add_symptom/1 - assert a symptom into working memory */
add_symptom(S) :-
    possible_symptom(S),
    ( symptom(S) -> true ; assertz(symptom(S)) ).

/* reset_case/0 - clear all asserted symptoms and derived facts
before starting a new consultation */
reset_case :-
    retractall(symptom(_)),
    retractall(derived(_)).

/* load_case/1 - load a list of symptoms at once, e.g.
?- load_case([fever, cough, body_pain]). */
load_case(SymptomList) :-
    reset_case,
    forall(member(S, SymptomList), add_symptom(S)).

/* explain/1 - simple explanation facility: shows which rule
proved a disease ("how" explanation) */
explain(flu) :-
    writeln('flu is concluded because: fever AND cough AND body_pain are all true.').

explain(covid19) :-
    writeln('covid19 is concluded because: high_fever AND dry_cough AND loss_of_taste AND
breathlessness are all true.').
explain(common_cold) :-
    writeln('common_cold is concluded because: runny_nose AND sore_throat are true, and high_fever is
absent.').
explain(dengue) :-
    writeln('dengue is concluded because: high_fever AND rashes AND joint_pain are all true.').
explain(malaria) :-
    writeln('malaria is concluded because: fever AND chills AND sweating are all true.').
explain(migraine) :-
    writeln('migraine is concluded because: headache AND nausea AND vomiting are all true.').

/* =====
END OF KNOWLEDGE BASE
===== */

```